

Linux pour l'admin cloud

Jour 3 — Automatiser

Bloc CL-LINUX (jour 8 du cursus) — scripts bash, grep/awk/sed, cron
et timers,
disques et réseau : la machine travaille pendant que vous dormez

Objectifs de la journée

À la fin de cette journée, vous serez capable de :

- Écrire un **script bash robuste** : arguments, variables, conditions, boucles, codes retour.
- Maîtriser les **redirections** (`>`, `>>`, `2>&1`, `<`) et les **pipes** (`|`).
- Extraire l'information utile des logs et des configs avec **grep, awk, sed**.
- **Planifier** une tâche avec **cron** ET avec un **timer systemd** — et choisir entre les deux.
- Diagnostiquer un **disque** : `lsblk`, `df -h`, `du -sh`, `/etc/fstab`, lire une pile LVM.
- Vérifier le **réseau local** d'un serveur : `ip a`, `ip r`, `ss -tlnp`, `ping`, `curl`, `dig`.

Le fil conducteur du cursus commence aujourd'hui pour de bon :

automatiser — un admin cloud n'exécute pas deux fois la même commande à la main.

Plan de la journée

1. **Scripting bash** — votre premier outil d'automatisation (~2 h 15) —  démo 3-1 construite au fil de la section.
2. **grep, awk, sed** — la boîte à outils texte de l'admin (~45 min).
3. **cron et timers systemd** — planifier sans y penser (~45 min) —  exercice 3-1.
4. **Disques et systèmes de fichiers** — de `lsblk` à LVM, et le lien avec EBS (~1 h).
5. **Outils réseau locaux** — le diagnostic express (~45 min) —  exercice 3-2.
6. Récap + quiz de fin de journée (~30 min).

1. Scripting bash

Écrire des scripts qui tiennent la route

Votre premier script : shebang et chmod +x

Un script = un fichier texte qui enchaîne des commandes. Sur la VM `srv-linux` :

```
nano bonjour.sh
```

```
#!/usr/bin/env bash
# bonjour.sh - mon premier script
echo "Bonjour $(whoami), nous sommes le $(date +%F)"
```

```
chmod +x bonjour.sh    # le droit x du jour 1, appliqué à votre fichier
./bonjour.sh           # ./ car le répertoire courant n'est pas dans le PATH
```

- Le **shebang** `#!/usr/bin/env bash` (ligne 1) indique quel interpréteur exécute le fichier.
- **Rappel cloud** : le *user data* d'une instance EC2, c'est exactement ça — un script exécuté au premier démarrage. Ce que vous écrivez aujourd'hui tournera là-bas tel quel.

Variables et quoting : la source n°1 de bugs

```
PRENOM="Awa"           # pas d'espace autour du = !  
echo "Bonjour $PRENOM" # guillemets doubles : $PRENOM est remplacé  
echo 'Bonjour $PRENOM' # guillemets simples : texte littéral  
JOUR=$(date +%A)       # $( ) capture la sortie d'une commande  
echo "Nous sommes $JOUR"
```

Le piège des espaces — **toujours quoter les variables** :

```
FICHIER="rapport du jour.txt"  
touch "$FICHIER"      # crée UN fichier  
rm $FICHIER           # ERREUR : rm reçoit 4 arguments (rapport, du, jour.txt)  
rm "$FICHIER"         # correct
```

Règle d'or : "\$VARIABLE" avec guillemets doubles, **partout, tout le temps**.

Les arguments : \$1, \$2, \$@, \$#

Un script utile est **paramétrable** — comme les commandes que vous utilisez déjà.

```
#!/usr/bin/env bash
# args.sh - inspecter les arguments reçus
echo "Nom du script      : $0"
echo "Premier argument   : $1"
echo "Deuxième argument  : $2"
echo "Nombre d'arguments : $#"
```

```
echo "Tous les arguments : $@"
```

```
./args.sh /etc /var/backups
```

Valeur **par défaut** si l'argument est absent — le motif à retenir :

```
SOURCE_DIR="${1:-/etc}"    # $1 s'il existe, sinon /etc
```

Codes retour : \$?, exit — et les enchaînements && / ||

Toute commande renvoie un code retour : `0` = succès, autre chose = erreur.

```
ls /etc > /dev/null ; echo $?    # 0 : succès
ls /inexistant      ; echo $?    # 2 : erreur
```

`&&` et `||` s'appuient dessus pour enchaîner :

```
sudo apt update && sudo apt upgrade -y    # la 2e SEULEMENT si la 1re réussit
grep -q "^stockline:" /etc/passwd || echo "utilisateur absent"
```

- Dans un script, `exit 1` termine avec le code 1 (que le lanceur lira dans `$?`).
- C'est le langage commun de l'automatisation : systemd, cron et vos futurs pipelines CI/CD décident **uniquement** d'après ce code.

Conditions : test, [] et if/else

```
#!/usr/bin/env bash
if [ -d "$1" ]; then
    echo "Le répertoire $1 existe"
else
    echo "ERREUR : répertoire $1 introuvable" >&2
    exit 1
fi
```

Test	Vrai si	Test	Vrai si
<code>-f fichier</code>	fichier ordinaire existe	<code>"\$a" = "\$b"</code>	chaînes égales
<code>-d rep</code>	répertoire existe	<code>-z "\$a"</code>	chaîne vide
<code>-w rep</code>	inscriptible	<code>"\$n" -lt 5</code>	$n < 5$ (<code>-eq</code> , <code>-le</code> , <code>-gt</code> , <code>-ge</code>)

- `[` est une **commande** (alias de `test`) : les **espaces sont obligatoires** après `[` et avant `]`.
- `>&2` envoie le message sur la sortie d'**erreur** — on y revient dans 3 slides.

Boucles : for (une liste) et while (une condition)

for parcourt une liste — fichiers, serveurs, arguments :

```
for fichier in /var/log/*.log; do
    echo "journal trouvé : $fichier"
done
```

while répète tant qu'une condition est vraie — attendre, réessayer :

```
n=1
while [ "$n" -le 5 ]; do
    echo "tentative $n"
    n=$((n + 1))      # $(( )) : arithmétique entière
done
```

Cas d'usage admin : **for** pour traiter 50 fichiers de logs ou 10 serveurs ;

while pour attendre qu'un service réponde avant de continuer un déploiement.

stdin, stdout, stderr : les trois flux de toute commande

```

+-----+
(0) stdin  -----> |  commande  | ----> (1) stdout : sortie normale
   clavier  | (processus) | ----> (2) stderr : messages d'erreur
+-----+
                        (par défaut, 1 et 2 vont au terminal)

```

```

cmd  > f      stdout ECRASE le fichier f
cmd >> f      stdout AJOUTE à la fin de f
cmd 2> f      stderr vers f (stdout reste au terminal)
cmd  > f 2>&1  stdout ET stderr vers f (2 rejoint 1)
cmd  < f      stdin lu depuis f au lieu du clavier
cmd1 | cmd2   stdout de cmd1 devient stdin de cmd2

```

Deux flux de sortie **séparés** : on peut garder le résultat et jeter
(ou journaliser) les erreurs — indispensable pour les tâches planifiées.

Redirections et pipes en pratique

```
echo "rapport du $(date +%F)" > rapport.txt    # crée ou écrase
echo "espace disque OK" >> rapport.txt        # ajoute à la fin
sudo apt update > /tmp/maj.log 2>&1          # TOUT capturer (réflexe cron)
wc -l < /etc/passwd                          # stdin depuis un fichier
```

Le pipe `|` branche des commandes bout à bout :

```
journalctl -u ssh | grep -i "failed" | wc -l  # compter les échecs SSH
ps aux | grep nginx                           # retrouver un processus
history | tail -20                             # vos 20 dernières commandes
```

Philosophie Unix : des petits outils qui font une chose bien, **assemblés par des pipes** — c'est comme ça qu'on construit un diagnostic en une ligne.

Fil rouge : backup.sh — v1, la version naïve

Objectif de la section : un script de sauvegarde digne d'un serveur de production.

Version 1 — ça marche, mais tout est en dur :

```
#!/usr/bin/env bash
# backup.sh v1 - archive /etc, tout est codé en dur
HORODATAGE=$(date +%Y%m%d-%H%M%S)
sudo mkdir -p /var/backups/demo
sudo tar -czf "/var/backups/demo/etc-${HORODATAGE}.tar.gz" /etc
echo "Sauvegarde terminée : etc-${HORODATAGE}.tar.gz"
```

Ce qui ne va pas :

- impossible de sauvegarder **autre chose** que `/etc` sans éditer le script ;
- aucune vérification : si `/etc` ou la destination manque, ça casse en silence ;
- les archives **s'accumulent** jusqu'à remplir le disque (panne classique !).

backup.sh — v2 : arguments et valeurs par défaut

```
#!/usr/bin/env bash
# backup.sh v2 - source et destination paramétrables
SOURCE_DIR="${1:-/etc}"          # argument 1, défaut /etc
DEST_DIR="${2:-/var/backups/demo}" # argument 2, défaut /var/backups/demo
HORODATAGE=$(date +%Y%m%d-%H%M%S)
NOM=$(basename "$SOURCE_DIR")    # /opt/app -> app
mkdir -p "$DEST_DIR"
tar -czf "${DEST_DIR}/${NOM}-${HORODATAGE}.tar.gz" "$SOURCE_DIR"
echo "[$(date '+%F %T')] sauvegarde de $SOURCE_DIR vers $DEST_DIR : OK"
```

```
sudo ./backup.sh          # les défauts
sudo ./backup.sh /home/ubuntu /tmp/sauvegardes # paramétré
```

Mieux. Mais si `tar` échoue à 3 h du matin, **personne ne le saura** — v3.

backup.sh — v3 : robuste (mode strict, rétention, codes retour)

```
#!/usr/bin/env bash
set -euo pipefail          # stop à la 1re erreur, variable non définie, pipe cassé
SOURCE_DIR="${1:-/etc}"
DEST_DIR="${2:-/var/backups/demo}"
RETENTION=7
if [ ! -d "$SOURCE_DIR" ]; then
    echo "[$(date '+%F %T')] ERREUR : source $SOURCE_DIR absente" >&2
    exit 1
fi
mkdir -p "$DEST_DIR"
tar -czf "${DEST_DIR}/${(basename "$SOURCE_DIR")}-${(date +%Y%m%d-%H%M%S)}.tar.gz" "$SOURCE_DIR"
ls -1t "$DEST_DIR"/*.tar.gz | tail -n +$((RETENTION + 1)) | xargs -r rm --
echo "[$(date '+%F %T')] OK : $(ls "$DEST_DIR" | wc -l) archives conservées"
```

Version complète en démo : + code retour **2** si la destination n'est pas inscriptible. Codes : **0** OK, **1** source absente, **2** destination non inscriptible.

Démo 3-1 — Le script de sauvegarde, en direct (35 min)

Fichier : `demos/03-cl-linux/demo-3-1-script-sauvegarde.md`

Construction de `backup.sh` en 3 versions, exactement comme dans les slides :

- **v1** : `tar` en dur — on constate les limites ;
- **v2** : arguments `$1/``$2` avec défauts, horodatage, messages ;
- **v3** : `set -euo pipefail`, rétention 7 archives, codes retour 0/1/2 ;
- **tests des cas d'erreur** : source absente, destination non inscriptible, vérification de `$?` après chaque essai ;
- **planification** avec `backup.timer` + `systemctl list-timers` (teaser section 3).

Le fichier final est `code/03-cl-linux/backup.sh` : vous le **réutiliserez demain** pour sauvegarder les données de StockLine, puis au CL-TP1.

2. grep, awk, sed

La boîte à outils texte de l'admin

grep : fouiller les logs (le réflexe n°1)

Sous Linux, tout est texte : logs, configs, sorties de commandes. `grep` filtre.

```
grep -i error /var/log/syslog          # -i : ignore la casse
grep -c "session opened" /var/log/auth.log  # -c : compter seulement
sudo grep -r "Failed password" /var/log/    # -r : récursif dans un dossier
sudo grep -riE "error|failed|critical" /var/log/nginx/  # -E : regex étendue
grep -v "^#" /etc/ssh/sshd_config          # -v : inverser (sans commentaires)
```

Le combo de l'admin : `-riE` = récursif + insensible à la casse + plusieurs motifs.

Rappel cloud : les logs sont la boîte noire du serveur. CloudWatch Logs

(bloc CL-AWS2) fera la même chose en dashboard — le filtre `grep` reste le même dans votre tête.

awk : extraire des colonnes — et l'assembler avec grep

`awk '{print $N}'` extrait la N-ième colonne (séparateur : les espaces).

```
df -h | awk '{print $5, $6}'      # % d'occupation + point de montage
ps aux | awk '{print $2, $11}'    # PID + commande
```

Exemple assemblé, lu de gauche à droite :

```
grep " 500 " access.log | awk '{print $1}' | sort | uniq -c | sort -rn
-----
garde les lignes des      extrait la 1re      trie, compte les doublons,
erreurs HTTP 500          colonne (IP client)  classe du plus fréquent

Résultat : les adresses IP qui provoquent le plus d'erreurs 500
42 203.0.113.7
3 198.51.100.12
```

Une ligne, quatre outils : c'est un **rapport d'incident** instantané.

sed -i : modifier une config sans ouvrir d'éditeur

`sed 's/ancien/nouveau/'` remplace du texte. Essayez sur la VM :

```
echo "port=8000" > app.conf
sed 's/8000/9000/' app.conf      # aperçu à l'écran, fichier INTACT
sed -i.bak 's/8000/9000/' app.conf # -i : modifie le fichier, .bak = copie
cat app.conf app.conf.bak       # port=9000 / port=8000
```

Bonnes pratiques :

- **toujours** un aperçu sans `-i` d'abord, puis `-i.bak` pour garder l'original ;
- indispensable dans les scripts : pas d'éditeur interactif dans un *user data* EC2 ni dans un pipeline — `sed -i` est la façon de modifier une config automatiquement.

Demain, vous l'utiliserez sur `sshd_config` ; les trois outils : `grep` trouve, `awk` découpe, `sed` modifie. Les cas d'usage vus ici couvrent 90 % du besoin.

3. cron et timers systemd

Planifier sans y penser

cron : la syntaxe des 5 champs

```
crontab -e    # éditer SA table de planification (une par utilisateur)
crontab -l    # lister ses tâches planifiées
```

```
* * * * *  commande
|   |   |   |   |
|   |   |   |   +--- jour de semaine (0-7 ; 0 et 7 = dimanche, 1 = lundi)
|   |   |   +----- mois                (1-12)
|   |   +----- jour du mois            (1-31)
|   +----- heure                      (0-23)
+----- minute                        (0-59)
```

```
0 3 * * *    tous les jours à 03h00
*/10 * * * * toutes les 10 minutes
0 4 * * 1     le lundi à 04h00
```

Les pièges de cron (tout le monde tombe dedans)

cron exécute vos tâches dans un environnement **minimal** — pas le vôtre :

- **PATH** réduit (`/usr/bin:/bin`), pas de `.bashrc` chargé →
chemins absolus obligatoires : `/usr/local/bin/backup.sh`, `/usr/bin/python3` ;
- pas de terminal : la sortie est perdue (ou part en mail local) →
redirigez tout — le motif canonique :

```
0 3 * * * /usr/local/bin/backup.sh >> /var/log/backup.log 2>&1
```

- le caractère `%` a un sens spécial dans une ligne cron (à échapper : `\%`) ;
- machine **éteinte à l'heure prévue** = tâche perdue, pas de rattrapage.

Diagnostic : `grep CRON /var/log/syslog` — cron y trace chaque lancement.

Ces limites ont une solution moderne : les **timers systemd**.

Timers systemd : la paire .service / .timer

Deux fichiers dans `/etc/systemd/system/`, même nom, extensions différentes :

```
# backup.service - QUOI faire (une unité comme hier, Type=oneshot)
[Unit]
Description=Sauvegarde quotidienne

[Service]
Type=oneshot
ExecStart=/usr/local/bin/backup.sh /etc /var/backups/demo
```

```
# backup.timer - QUAND le faire
[Unit]
Description=Declenche backup.service chaque nuit

[Timer]
OnCalendar=*-*-* 03:00:00
Persistent=true

[Install]
WantedBy=timers.target
```

`OnCalendar=*-*-* 03:00:00` : tous les jours à 03h00.

`Persistent=true` : machine éteinte à 03h00 → **rattrapage au démarrage suivant**.

Activer, superviser — et choisir : cron ou timer ?

```
sudo systemctl daemon-reload
sudo systemctl enable --now backup.timer    # on active le TIMER, pas le service
systemctl list-timers                       # NEXT, LAST, unité déclenchée
journalctl -u backup.service                # les journaux de la tâche
```

Critère	cron	timer systemd
Mise en place	1 ligne (<code>crontab -e</code>)	2 fichiers
Journaux	à rediriger soi-même	intégrés (<code>journalctl -u</code>)
Machine éteinte à l'heure dite	tâche perdue	rattrapée (<code>Persistent=true</code>)
Supervision	<code>grep CRON /var/log/syslog</code>	<code>systemctl list-timers</code>
Dépendances (réseau, montage)	non	oui (mécanique systemd)

En pratique : cron pour la ligne vite posée, **timer pour tout ce qui compte** —
sur vos EC2, la sauvegarde de StockLine sera un timer.

Exercice 3-1 — Script de nettoyage de logs (60 min)

Fichier : `exercices/03-cl-linux/exercice-3-1-nettoyage-logs.md`

Une application fictive remplit `/var/log/appdemo` (bloc de mise en place fourni, à copier-coller). Vous écrivez `nettoie-logs.sh` :

- **arguments** : répertoire cible et âge en jours, avec valeurs par défaut ;
- **compresser** (gzip) les `.log` de plus de 1 jour ;
- **supprimer** les `.log.gz` plus vieux que N jours ;
- compter et afficher ce qui a été traité, **codes retour** propres ;
- **planifier** l'exécution chaque nuit à 04h00 avec cron (`0 4 * * *`).

Bonus : la version timer systemd (paire `.service` / `.timer`).

Objectif : réinvestir TOUTE la matinée — arguments, tests, boucles, redirections, planification. C'est le script type d'un serveur en production.

4. Disques et systèmes de fichiers

De lsblk à LVM — et à EBS

La pile de stockage, et lsblk pour la voir

```

disque physique    /dev/sda    (sur EC2 : le volume EBS attaché)
|
partition         /dev/sda1   (découpage logique du disque)
|
système de fichiers ext4      (l'organisation des données : mkfs)
|
point de montage  /           (le répertoire d'accès : mount, fstab)

```

```
lsblk           # la pile, vue depuis la VM
```

```

NAME      SIZE TYPE MOUNTPOINTS
sda        20G disk
|-sda1    18,9G part /
|-sda15   106M part /boot/efi
`-sda16   913M part /boot

```

EBS = le disque que vous verrez dans `lsblk` sur vos instances
(nommé `xvda` ou `nvme0n1` selon le type d'instance).

df -h et du -sh : « le disque est plein » vs « qu'est-ce qui le remplit ? »

```
df -h                # occupation de chaque FS monté
df -h /              # juste la racine
du -sh /var/log      # poids total d'un répertoire (-s résumé)
sudo du -h --max-depth=1 /var | sort -h # le coupable, répertoire par répertoire
```

Commande	Question à laquelle elle répond	Point de vue
<code>df -h</code>	« Reste-t-il de la place ? »	le système de fichiers
<code>du -sh chemin</code>	« Combien pèse cette arborescence ? »	le répertoire

Méthode de diagnostic « disque plein » (alerte la plus fréquente du métier) :

`df -h` pour trouver le FS saturé → `du -h --max-depth=1` en descendant l'arborescence jusqu'au coupable (souvent `/var/log`... d'où l'exercice 3-1).

Monter un disque : mount, umount et /etc/fstab champ par champ

```
sudo mkdir -p /data
sudo mount /dev/sdb1 /data    # montage manuel : PERDU au redémarrage
findmnt /data                # vérifier ce qui est monté là
sudo umount /data            # démonter (échoue si un fichier y est ouvert)
```

Pour monter à chaque démarrage : une ligne dans `/etc/fstab` –

```
/dev/sdb1  /data  ext4  defaults  0  2
|          |    |    |          |  +- passe fsck (0 jamais, 1 racine, 2 autres)
|          |    |    |          |  +----- dump (sauvegarde historique : 0)
|          |    |    |          |  +----- options de montage (defaults = rw, async, exec)
|          |    |    |          |  +----- type de système de fichiers
|          |    |    |          |  +----- point de montage
|          |    |    |          |  +----- périphérique (en prod : UUID= donné par blkid)
```

Piège sérieux : une faute dans fstab peut **bloquer le démarrage**.

Toujours valider avec `sudo mount -a` (monte tout fstab) AVANT de redémarrer.

LVM (1/2) : PV, VG, LV — le stockage devient souple

Problème des partitions classiques : tailles **figées** à la création.

LVM (*Logical Volume Manager*) insère une couche d'abstraction :

```

partitions/disques  /dev/sdb1    /dev/sdc1    <- PV (Physical Volume)
                    \          /
groupe de volumes   vg_data      <- VG (le "pot commun")
                    /          \
volumes logiques    lv_app (20G)  lv_logs (10G) <- LV (découpés à la demande)
                    |            |
FS + montage        ext4 /var/app  ext4 /var/log/app
  
```

- **PV** : un disque ou une partition versé au pot ;
- **VG** : le pot commun qui agrège la capacité de tous les PV ;
- **LV** : la « partition souple » qu'on découpe dans le VG — c'est elle qu'on formate et qu'on monte.

LVM (2/2) : ce que ça permet — et le parallèle EBS

Pas de manipulation aujourd'hui — savoir **lire** une pile LVM suffit :

```
sudo pvs    # les Physical Volumes (lecture seule)
sudo vgs    # les Volume Groups et leur espace libre
sudo lvs    # les Logical Volumes et leurs tailles
```

Ce que LVM permet (d'où sa présence sur presque tous les serveurs d'entreprise) :

- **agrandir un volume à chaud** (`lvextend`), sans démonter ni redémarrer ;
- ajouter un disque au VG quand l'espace manque ;
- instantanés (*snapshots*) avant une opération risquée.

Parallèle cloud : sur AWS, **EBS rend le même service à sa façon** — on

agrandit un volume depuis la console/API, puis on étend le FS dans la VM.

Vous verrez LVM sur l'on-premise et certaines images (RHEL l'utilise par défaut) ;

dans `lsblk`, une pile LVM apparaît avec `TYPE=lv` sous la partition.

5. Outils réseau locaux

**Le diagnostic express — avant le grand bain
CL-RÉSEAU**

ip a et ip r : qui suis-je, par où je sors ?

```
ip a      # adresses : cherchez "inet" sur votre interface
ip -br a  # version brève, très lisible
ip r      # table de routage : la ligne "default" = la passerelle
```

Sortie type sur la VM Multipass (interface `ens3` ou `enp0s1` selon l'hyperviseur) :

```
lo        UNKNOWN    127.0.0.1/8
ens3      UP         192.168.64.5/24
```

```
default via 192.168.64.1 dev ens3
192.168.64.0/24 dev ens3 proto kernel scope link
```

- C'est l'IP que `multipass info srv-linux` vous affiche depuis l'hôte.
- **Rappel cloud** : sur EC2, `ip a` montre l'IP **privée** ; l'IP publique vit dans le réseau AWS (NAT), la VM ne la connaît pas. Le `/24` ? Réponse au bloc CL-RÉSEAU.

ss -tlnp : qui écoute sur quel port ?

LA commande pour savoir quels services attendent des connexions :

```
sudo ss -tlnp # -t TCP, -l en écoute, -n ports numériques, -p processus
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	4096	0.0.0.0:22	0.0.0.0:*	sshd
LISTEN	0	511	0.0.0.0:80	0.0.0.0:*	nginx
LISTEN	0	2048	127.0.0.1:8000	0.0.0.0:*	uvicorn

Lire la colonne *Local Address* :

- **0.0.0.0:80** : écoute sur **toutes** les interfaces — joignable de l'extérieur (si le pare-feu / security group l'autorise) ;
- **127.0.0.1:8000** : **local uniquement** — c'est exactement le montage nginx → uvicorn du déploiement StockLine de demain.

ping, curl, dig : la machine, le service, le nom

Trois questions différentes, trois outils :

```
ping -c 3 ubuntu.com      # la MACHINE répond-elle ? (ICMP)
curl -I https://ubuntu.com # le SERVICE web répond-il ? (-I : en-têtes seuls)
curl -s http://127.0.0.1:8000/sante # -s silencieux : parfait dans un script
dig +short ubuntu.com      # le NOM se résout-il ? (réponse DNS brute)
```

Le raisonnement de dépannage à ancrer :

- **dig** échoue → problème **DNS** ;
- **ping** échoue → machine injoignable (réseau, machine éteinte) ;
- **ping** OK mais **curl** échoue → machine vivante, **service** arrêté ou **port filtré** (pare-feu... ou security group sur AWS).

Chaque étape isole une couche — vous formaliserez tout ça au bloc CL-RÉSEAU.

Exercice 3-2 – Audit disque et réseau d'un serveur (45 min)

Fichier : `exercices/03-cl-linux/exercice-3-2-audit-disque-reseau.md`

Mise en situation : on vous confie un serveur « qui se remplit » – le bloc de mise en place crée des fichiers cachés bien gras. Votre mission d'audit :

- retrouver **ce qui remplit le disque** : `df -h`, puis `du -sh /`
`du -h --max-depth` en descendant jusqu'au coupable ;
- lire la topologie : `lsblk` et `/etc/fstab` ;
- relever **IP et route** (`ip a`, `ip r`), les **ports en écoute** (`ss -tlnp`) ;
- tester un service web (`curl -I`) et résoudre un nom (`dig +short`) ;
- restituer le tout dans un **mini-rapport texte** – le livrable type qu'on attend d'un admin après un audit.

Objectif : dérouler seul la méthode de diagnostic complète des sections 4 et 5.

Récap des points clés du jour

- Un script bash : **shebang** + `chmod +x` ; variables **toujours quotées** ; arguments `$1`/`$@`/`$#` avec défauts `${1:-valeur}`.
- **Code retour** : `0` = succès — la base de `if`, `&&`, `||` et de toute l'automatisation. `set -euo pipefail` dans tout script sérieux.
- Redirections : `>` écrase, `>>` ajoute, `2>&1` capture aussi les erreurs, `|` assemble les outils.
- **grep** trouve (`-riE`), **awk** découpe (`{print $N}`), **sed** modifie (`-i`).
- Planification : cron = 5 champs + pièges (PATH, sortie) ; timer systemd = paire `.service`/`.timer`, journaux intégrés, `Persistent=true`.
- Disques : `lsblk` (la pile), `df -h` (le FS), `du -sh` (le répertoire), `/etc/fstab` (montages au boot), LVM = PV → VG → LV. **EBS = votre futur disque.**
- Réseau local : `ip a`/`ip r`, `ss -tlnp`, puis `dig` → `ping` → `curl` pour isoler la panne.



Quiz — questions 1 et 2

Question 1

Que contient la variable spéciale `$?` juste après une commande ?

Question 2

Quelle est la différence entre `>` et `>>` ?



Quiz — questions 3 et 4

Question 3

Que fait `2>&1` à la fin d'une commande ?

Question 4

Que calcule `grep -i error /var/log/syslog | wc -l` ?



Quiz — questions 5 et 6

Question 5

La ligne cron `0 3 * * 1` : quand la tâche s'exécute-t-elle ?

Question 6

Citez deux avantages d'un timer systemd sur une ligne cron.



Quiz — questions 7 et 8

Question 7

Quelle est la différence entre `df -h` et `du -sh /var/log` ?

Question 8

À quoi sert le fichier `/etc/fstab` ?



Quiz — questions 9 et 10

Question 9

Que montre `ss -tlnp` ?

Question 10

`ping` répond mais `curl http://serveur` échoue : que peut-on en conclure ?

Jour 4 — SSH, sécurité et LE déploiement de StockLine

Aujourd'hui, la machine a appris à travailler seule. Demain, vous **ouvrez,**
sécurisez et déployez : SSH par clés, durcissement de `sshd`, pare-feu `ufw` —
puis le grand moment : StockLine en production sur votre VM (nginx +
uvicorn +
systemd), sauvegardée chaque nuit par le `backup.sh` d'aujourd'hui.